

電腦視覺輕鬆上手：Spring Boot 和 Java 適用的 Vision AI

來源：<https://codelabs.developers.google.com/cloudvision-translate-springboot?hl=zh-tw>

步驟數：7

我們會使用 Spring Boot 和 Java 建立電腦視覺應用程式，讓您在專案中充分發揮圖片辨識與分析的潛能。

目錄

1. [1. 簡介](#)
2. [2. 需求條件](#)
3. [3. 啟動 Spring Boot 專案](#)
4. [4. 設定 Vision API](#)
5. [5. 在 Cloud Run 上部署電腦視覺應用程式](#)
6. [6. 清除所用資源](#)
7. [7. 恭喜](#)

1. 簡介

在當今資料驅動的應用程式時代，運用電腦視覺等進階機器學習和人工智慧服務變得越來越重要。其中一項服務是 [Vision API](#)，可提供強大的圖片分析功能。在本程式碼研究室中，您將瞭解如何使用 Spring Boot 和 Java 建立 Computer Vision 應用程式，在專案中發揮圖像辨識和分析的潛力。應用程式 UI 會接受含有手寫或印刷文字的圖片公開網址做為輸入內容，然後擷取文字、偵測語言，如果該語言是 [支援](#) 的語言之一，就會生成該文字的英文翻譯。

建構項目

您將建立

- 使用 Vision API 和 Google Cloud Translation API 的 Java Spring Boot 應用程式
 - 部署在 Cloud Run 上
-

步驟 2 · 約 0 分鐘

2. 需求條件

- [Chrome](#) 或 [Firefox](#) 瀏覽器
- 已啟用計費功能的 Google Cloud 專案

事前須知如下：

建立專案

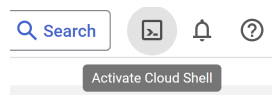
1. 已建立 [專案](#) 並啟用 [帳單](#) 的 [Google Cloud](#) 帳戶
2. 已啟用 Vision API、Translation、Cloud Run 和 Artifact Registry API
3. [Cloud Shell](#) 已啟用
4. 已啟用 Cloud Storage API、已建立值區，且已上傳含有以當地支援語言撰寫的文字或手寫內容的圖片 (您也可以使用這篇網誌提供的範例圖片連結)

如需啟用 Google Cloud API 的步驟，請參閱[說明文件](#)。

啟用 Cloud Shell

1. 您將使用 [Cloud Shell](#)，這是 Google Cloud 中執行的指令列環境，已預先載入 **bq**：

在 Cloud 控制台，按一下右上角的「啟用 Cloud Shell」



2. 連至 Cloud Shell 後，您應該會看到驗證已完成，專案也已設為獲派的專案 ID。在 Cloud Shell 中執行下列指令，確認您已通過驗證：

〇〇〇〇

```
gcloud auth list
```

3. 在 Cloud Shell 中執行下列指令，確認 gcloud 指令已瞭解您的專案

〇〇〇〇

```
gcloud config list project
```

4. 如果未設定專案，請使用下列指令來設定：

〇〇〇〇

```
gcloud config set project <PROJECT_ID>
```

如要瞭解 gcloud 指令和用法，請參閱[說明文件](#)。

步驟 3 · 約 0 分鐘

3. 啟動 Spring Boot 專案

如要開始使用，請使用偏好的 IDE 或 Spring Initializr 建立新的 Spring Boot 專案。在專案設定中加入必要的依附元件，例如 Spring Web、Spring Cloud GCP 和 Vision AI。或者，您也可以按照下列步驟，使用 Cloud Shell 中的 Spring Initializr 輕鬆啟動 Spring Boot 應用程式。

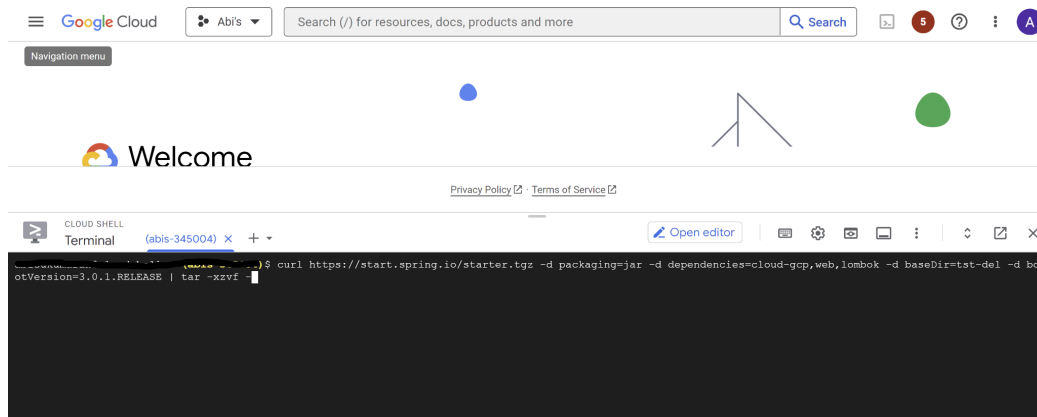
執行下列指令，建立 Spring Boot 專案：

```
curl https://start.spring.io/starter.tgz -d packaging=jar -d dependencies=cloud-gcp,web,lombok -d baseDir=spring-vision -d type=maven-project -d bootVersion=3.0.1.RELEASE | tar -xzf -
```

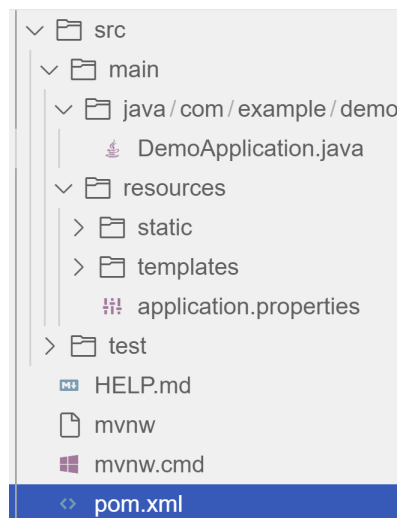
spring-vision 是專案名稱，請視需要變更。

bootVersion 是 Spring Boot 的版本，請務必在實作時視需要更新。

type 是專案建構工具類型的版本，您可以視需要將其變更為 **gradle**。



這會在「spring-vision」底下建立專案結構，如下所示：



pom.xml 包含專案的所有依附元件 (您使用這個指令設定的依附元件已新增至 **pom.xml**)。

src/main/java/com/example/demo 含有來源類別 **.java** 檔案。

資源包含專案使用的圖片、XML、文字檔和靜態內容，且這些資源是獨立維護。

application.properties 可讓您維護管理員功能，定義應用程式的設定檔專屬屬性。

4. 設定 Vision API

啟用 Vision API 後，您可以選擇在應用程式中設定 API 憑證。您也可以選擇使用應用程式預設憑證設定驗證。不過，在這個示範實作中，我並未實作憑證的使用。

實作 Vision 和翻譯服務

建立與 Vision API 互動的服務類別。插入必要依附元件，並使用 Vision API 用戶端傳送圖片分析要求。您可以根據應用程式需求，實作執行圖像標籤、臉部偵測、辨識等工作的相關方法。在本示範中，我們將使用手寫文字擷取和翻譯方法。

為此，請務必在 pom.xml 中加入下列依附元件。

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-gcp-starter-vision</artifactId>
</dependency>
```

```
<dependency>
  <groupId>com.google.cloud</groupId>
  <artifactId>google-cloud-translate</artifactId>
</dependency>
```

從 [存放區](#) 複製 / 取代下列檔案，然後將檔案新增至專案結構中的相應資料夾 / 路徑：

1. Application.java (/src/main/java/com/example/demo)
2. TranslateText.java (/src/main/java/com/example/demo)
3. VisionController.java (/src/main/java/com/example/demo)
4. index.html (/src/main/resources/static)
5. result.html (/src/main/resources/templates)
6. pom.xml

服務 `org.springframework.cloud.gcp.vision.CloudVisionTemplate` 中的 `extractTextFromImage` 方法可讓您從圖片輸入內容中擷取文字。服務 `com.google.cloud.translate.v3` 的 `getTranslatedText` 方法可讓您傳遞從圖片擷取的文字，並以所需目標語言取得翻譯文字做為回應 (前提是來源語言在[支援](#)的語言清單中)。

建構 REST API

設計及實作 REST 端點，公開 Vision API 功能。建立可處理傳入要求的控制器，並使用 Vision API 服務處理圖片及傳回分析結果。在本範例中，我們的 `VisionController` 類別會實作端點、處理傳入的要求、叫用 Vision API 和 Cloud Translation 服務，並將結果傳回檢視層。REST 端點的 GET 方法實作方式如下：

```
@GetMapping("/extractText")
public String extractText(String imageUrl) throws IOException {
    String textFromImage =
        this.cloudVisionTemplate.extractTextFromImage(this.resourceLoader.getResource(imageUrl));

    TranslateText translateText = new TranslateText();
    String result = translateText.translateText(textFromImage);
    return "Text from image translated: " + result;
}
```

上述實作中的 **TranslateText** 類別具有叫用 Cloud Translation 服務的方法：

```
String targetLanguage = "en";
TranslateTextRequest request =
    TranslateTextRequest.newBuilder()
        .setParent(parent.toString())
        .setMimeType("text/plain")
        .setTargetLanguageCode(targetLanguage)
        .addContents(text)
        .build();
TranslateTextResponse response = client.translateText(request);
// Display the translation for each input text provided
for (Translation translation : response.getTranslationsList()) {
    res = res + " ::: " + translation.getTranslatedText();
    System.out.printf("Translated text : %s\n", res);
}
```

透過 **VisionController** 類別，我們已實作 REST 的 GET 方法。

整合 Thymeleaf 以進行前端開發

使用 Spring Boot 建構應用程式時，前端開發人員通常會選擇運用 Thymeleaf 的強大功能。Thymeleaf 是伺服器端 Java 範本引擎，可讓您順暢地將動態內容整合至 HTML 網頁。Thymeleaf 可讓您建立內嵌伺服器端運算式的 HTML 範本，提供流暢的開發體驗。這些運算式可用於動態算繪 Spring Boot 後端的資料，方便顯示 Vision API 服務執行的圖片分析結果。

如要開始使用，請確認 Spring Boot 專案中已具備 Thymeleaf 的必要依附元件。您可以在 pom.xml 中加入 Thymeleaf Starter 依附元件：

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-thymeleaf</artifactId>
</dependency>
```

在控制器方法中，從 Vision API 服務擷取分析結果，並新增至模型。這個模型代表 Thymeleaf 用於算繪 HTML 範本的資料。模型填入資料後，請傳回要算繪的 Thymeleaf 範本名稱。Thymeleaf 會負責處理範本、將伺服器端運算式替換成實際資料，並產生最終 HTML，傳送至用戶端的瀏覽器。

以 **VisionController** 中的 **extractText** 方法為例，我們已將結果做為 **String** 傳回，但未新增至模型。但我們已在頁面提交時，對 **index.html** 叫用 GET 方法 **extractText** 方法。使用 Thymeleaf，您可以打造順暢的使用者體驗，

讓使用者上傳圖片、觸發 Vision API 分析，並即時查看結果。運用 Thymeleaf 進行前端開發，充分發揮 Vision AI 應用程式的潛力。

```
<form action="/extractText">
    Web URL of image to analyze:
    <input type="text"
        name="imageUrl"
        value=""
    >
    <input type="submit" value="Read and Translate" />
</form>
```

步驟 5 · 約 0 分鐘

5. 在 Cloud Run 上部署電腦視覺應用程式

請為服務和控制器類別撰寫單元測試，確保 `/src/test/java/com/example` 資料夾中的功能正常運作。確認穩定性後，請將其封裝至可部署的構件 (例如 JAR 檔案)，然後部署至 Google Cloud 的無伺服器運算平台 Cloud Run。在本步驟中，我們將著重於使用 Cloud Run 部署容器化 Spring Boot 應用程式。

1. 在 Cloud Shell 中執行下列步驟，將應用程式封裝(請確認終端機提示位於專案根資料夾)：

版本：

```
./mvnw package
```

建構成功後，請在本機執行測試：

```
./mvnw spring-boot:run
```

2. 使用 Jib 將 Spring Boot 應用程式容器化：

您可以使用 [Jib](#) 公用程式簡化容器化程序，不必手動建立 `Dockerfile` 並建構容器映像檔。Jib 是一個外掛程式，可直接與建構工具 (例如 Maven 或 Gradle) 整合，讓您建構最佳化的容器映像檔，不必編寫 `Dockerfile`。請先啟用 Artifact Registry API (建議使用 Artifact Registry，而非 Container Registry)，然後執行 Jib 建構 Docker 映像檔，並發布至 Registry：

```
$ ./mvnw com.google.cloud.tools:jib-maven-plugin:3.1.1:build -  
Dimage=gcr.io/$GOOGLE_CLOUD_PROJECT/vision-jib
```

注意：在本實驗中，我們並未在 `pom.xml` 中設定 Jib Maven 外掛程式，但如要進階使用，可以在 `pom.xml` 中新增該外掛程式，並設定更多選項

3. 將容器 (在上一步中推送至 Artifact Registry 的容器) 部署至 Cloud Run。這同樣是單一指令步驟：

```
gcloud run deploy vision-app --image gcr.io/$GOOGLE_CLOUD_PROJECT/vision-jib --platform  
managed --region us-central1 --allow-unauthenticated --update-env-vars
```

你也可以透過 UI 執行這項操作。前往 Google Cloud 控制台，然後找出 Cloud Run 服務。按一下「建立服務」，然後按照畫面上的指示操作。指定先前推送至登錄檔的容器映像檔、設定所需的部署設定 (例如 CPU 分配和自動調度資源)，然後選擇適當的部署區域。您可以設定應用程式專用的環境變數。這些變數可能包括驗證憑證 (API 金鑰等)、資料庫連線字串，或 Vision AI 應用程式正常運作所需的任何其他設定。部署作業成功完成後，您應該會取得應用程式的端點。

使用 Vision AI 應用程式

為進行示範，您可以讓應用程式讀取並翻譯以下圖片網址：

https://storage.googleapis.com/img_public_test/tamilwriting1.jfif



步驟 6 · 約 0 分鐘

6. 清除所用資源

如要避免系統向您的 Google Cloud 帳戶收取本文章所用資源的費用，請按照下列步驟操作：

1. 在 Google Cloud 控制台中前往「管理資源」頁面
 2. 在專案清單中選取要刪除的專案，然後按一下「刪除」
 3. 在對話方塊中輸入專案 ID，然後按一下「關閉」刪除專案
-

7. 恭喜

恭喜！您已成功使用 Spring Boot 和 Java 建立 Vision AI 應用程式。現在，應用程式可運用 Vision AI 執行複雜的圖片分析，包括加上標籤、偵測臉部等。整合 Spring Boot 後，您就能奠定穩固基礎，建構可擴充且穩健的 [Google Cloud Native](#) 應用程式。繼續探索 Vision AI、Cloud Run、Cloud Translation 等服務的強大功能，為應用程式新增更多特色和功能。詳情請參閱 [Vision API](#)、[Cloud Translation](#) 和 [GCP Spring](#) 文件。請嘗試使用 [Spring Native](#) 選項進行相同的實驗！此外，如要搶先一窺生成式 AI 世界，請查看這個 API 在 [模型園地](#) 中的顯示方式。
